SmartAgri

AI-Powered Precision Agriculture Advisory Platform

Project Report

Full System Architecture · Workflow · File Structure · Module Descriptions

Project Name:	SmartAgri AI Advisory Platform
Technology Stack:	100% Pure MERN Stack
Technologies:	MongoDB Atlas · Express.js · React 19 · Node.js
AI Integration:	Google Gemini Flash API
Weather API:	OpenWeatherMap
Languages Supported:	English · हिन्दी · தமிழ் · తెలుగు
Version:	1.0.0
Report Date:	October 2026

Table of Contents

1. 1. Project Concept and Objective
2. 2. System Architecture
3. 3. Data Flow and Request Lifecycle
4. 4. REST API Endpoints
5. 5. Full File Structure
6. 6. Core Modules in Detail
    1. 6.1 Authentication System
    2. 6.2 Gemini AI Chatbot
    3. 6.3 Crop Recommendation Engine
    4. 6.4 IoT Soil Sensor Telemetry
    5. 6.5 Weather Advisory Engine
    6. 6.6 3D Farm Visualizer
    7. 6.7 Multilingual i18n System
7. 7. Technology Stack
8. 8. Database Schema
9. 9. Multilingual Support
10. 10. End-to-End Application Workflow
11. 11. Environment Configuration
12. 12. Key Design Decisions
13. 13. Conclusion

1. Project Concept and Objective

SmartAgri is a full-stack, AI-driven digital agriculture advisory platform designed to empower Indian farmers with real-time, data-driven decision-making tools. The platform is built entirely on the MERN stack — MongoDB, Express.js, React 19, and Node.js — without relying on any third-party hosting service or external backend framework. It is accessible through any modern web browser and supports four Indian languages: English, Hindi (हिन्दी), Tamil (தமிழ்), and Telugu (తెలుగు), making it deeply inclusive for farmers across different regions of India.

The core problem this platform solves is the lack of affordable, real-time agronomic advisory for Indian smallholder farmers who typically own less than five acres of land. These farmers face daily challenges such as identifying the right crop to grow based on soil conditions, diagnosing plant diseases without visiting a government agricultural office, managing irrigation efficiently, and staying informed about government subsidy schemes and market prices. Without access to agricultural experts in their native language, many of them make uninformed decisions that lead to crop loss and financial hardship.

SmartAgri addresses all of these problems through a unified web application that brings together the following capabilities: IoT soil sensor telemetry simulation, a Google Gemini AI-powered multilingual chatbot, an algorithmic crop recommendation engine, a weather advisory system powered by OpenWeatherMap, a leaf disease detection tool, a fertilizer dose calculator, a smart irrigation planner, a live market price tracker, a government scheme navigator, and an interactive 3D farm visualizer built with CSS 3D transforms and Framer Motion animations.

> **Target Users:** Indian smallholder farmers (less than 5 acres), agriculture extension officers, and agricultural students seeking a smart advisory platform in their native language.

2. System Architecture

The SmartAgri platform follows a classic three-tier MERN architecture where the client layer, the server layer, and the database layer are clearly separated and communicate over well-defined HTTP REST APIs. Each layer has a clearly defined responsibility and can be independently scaled or replaced.



```
│                                                          │
│   Google Gemini Flash API    →  Multilingual AI Chatbot  │
│   OpenWeatherMap API         →  Live Weather + 7-Day Forecast │
└──────────────────────────────────────────────────────────┘
```

The Client Layer is built using React 19 and served via the Vite development server on port 5173. It is a Single Page Application (SPA) where all navigation between the eight feature modules happens without page reloads. The client communicates with the backend exclusively through HTTP REST API calls using the browser's native Fetch API.

The Server Layer is a Node.js application powered by Express.js, running on port 5000. Express acts as the API Gateway and routes every incoming request to the appropriate controller. Each controller encapsulates the business logic for its specific domain. The server handles CORS to allow the React frontend, running on a different port, to make cross-origin API calls. Multipart image uploads for leaf disease detection are handled by the Multer middleware.

The Database Layer uses MongoDB Atlas, a fully managed cloud-hosted MongoDB cluster. Mongoose serves as the Object Document Mapper (ODM) for MongoDB in Node.js. The connection is graceful — if Atlas is unreachable due to an IP access restriction, the application automatically falls back to an in-memory persistence mode and continues to run in demo mode without crashing.

3. Data Flow and Request Lifecycle

When a farmer opens the SmartAgri platform in their browser, the React application loads and immediately applies the farmer's selected language from the translation dictionary. All UI text — labels, button names, tab headings, card descriptions — is rendered in the chosen language.

AI Chatbot Flow: When the farmer types or speaks a question, the React component captures the message, attaches the current language code (such as "te" for Telugu), and sends it as a POST request to `/api/chat/ask`. The chatbot controller first attempts to call Google Gemini AI with a structured system prompt. If Gemini is available, the response is returned in the farmer's chosen language. If not, the controller falls back to its built-in keyword-matching agronomic knowledge base. Either way, the response is packaged with suggested follow-up questions and returned to the React client.

Weather Advisory Flow: When the farmer opens the Weather Advisory tab and enters a city name, the React component sends a GET request to `/api/weather/advisory`. The weather controller calls OpenWeatherMap's current weather and forecast endpoints, processes the data into agriculture-specific metrics, generates three smart advisory messages, and returns a seven-day farming forecast with spray/harvest suitability flags.

Authentication Flow: When a farmer registers or logs in, the React auth modal sends a POST request to `/api/auth/register` or `/api/auth/login`. The auth controller hashes the password using bcryptjs and stores the user in MongoDB Atlas, or verifies the credentials on login. On success, a JWT token is issued and returned to the React client, where it is stored in application state for use in subsequent authenticated requests.

IoT Telemetry Flow: The Farmer Dashboard polls `/api/iot/telemetry` on a fixed interval. The IoT controller updates the virtual sensor state with small random fluctuations to simulate realistic field sensor behavior and returns the current readings. If auto-mode is enabled, the pump is activated or deactivated automatically based on soil moisture thresholds.

4. REST API Endpoints

POST `/api/auth/register`

Register a new farmer account. Accepts name, email, password, phone, location, and farm details. Hashes password with bcrypt before storing in MongoDB.

POST `/api/auth/login`

Authenticate existing farmer. Verifies bcrypt password hash. Returns a signed JWT token valid for 24 hours.

GET `/api/auth/profile`

Retrieve authenticated farmer's profile. Requires Bearer JWT token in the Authorization header.

POST `/api/crop/recommend`

AI Crop Recommendation. Accepts Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, Rainfall, and Soil Type. Returns top 3 crops with confidence score using a 7-parameter weighted scoring algorithm.

GET `/api/weather/advisory`

Live weather data and 7-day agriculture forecast. Query params: city, state. Integrates OpenWeatherMap live API with smart farming advisories.

POST `/api/disease/detect`

Leaf Disease Detection. Accepts uploaded crop leaf image (multipart/form-data). Returns disease name, confidence, severity, and treatment plan with organic and chemical remedies.

POST `/api/irrigation/calculate`

Smart Irrigation Planner. Accepts crop type, field area, soil moisture, and weather data. Returns optimal drip or flood irrigation schedule with cycle count, duration, and total water volume.

POST `/api/fertilizer/recommend`

Fertilizer Dose Calculator. Accepts crop type, growth stage, and soil deficiencies. Returns balanced NPK fertilizer dosage split into basal and top-dress applications.

GET `/api/market/prices`

APMC Mandi Market Prices. Returns current wholesale price data for major Indian crops across key mandi locations.

POST `/api/market/predict-profit`

Seasonal Profit Prediction. Accepts crop, acreage, and expected yield. Returns projected profit forecast based on market price trends.

POST `/api/chat/ask`

Gemini AI Multilingual Chatbot. Accepts farmer message and language code (en/hi/ta/te). Calls Google Gemini Flash model with agriculture system prompt. Falls back to built-in knowledge base if Gemini is unavailable.

GET `/api/schemes`

Government Agriculture Schemes. Returns curated database of PM-KISAN, PM-KUSUM, PM Fasal Bima Yojana, Soil Health Card, and other central/state schemes with eligibility and application details.

GET `/api/iot/telemetry`

Real-time IoT Soil Sensor Telemetry. Returns current readings for soil moisture, temperature, NPK, pH, electrical conductivity, pump status, and auto-mode state with realistic fluctuations.

POST `/api/iot/pump`

Smart Pump Control. Accepts pumpActive (boolean) and autoMode (boolean) to manually toggle the water pump or switch between manual and automatic irrigation control.

GET `/health`

System Health Check. Returns server status, service name, timestamp, and MongoDB Atlas connection state.

5. Full File Structure

```
d:\Smartagri\ ← Project Root
│
├── package.json ← Root workspace configuration
├── README.md
│
├── server\ ← Node.js + Express Backend
│   ├── server.js ← Entry point: MongoDB connect, middleware, app bootstrap
│   ├── package.json ← Backend dependencies
│   ├── .env ← Secrets: MONGODB_URI, GEMINI_API_KEY, WEATHER_API_KEY
│   │
│   ├── routes\
│   │   └── api.js ← All 15 REST route declarations
│   │
│   ├── controllers\ ← Business Logic Layer
│   │   ├── authController.js ← Register, Login, JWT profile
│   │   ├── chatbotController.js ← Gemini AI + fallback agronomy engine
│   │   ├── cropController.js ← NPK scoring algorithm (9 crops)
│   │   ├── diseaseController.js ← Leaf disease image classification
│   │   ├── fertilizerController.js ← NPK fertilizer dose calculator
│   │   ├── iotController.js ← IoT sensor simulation + pump toggle
│   │   ├── irrigationController.js ← Smart irrigation scheduler
│   │   ├── marketController.js ← APMC prices + profit predictor
│   │   ├── schemeController.js ← Government scheme navigator
│   │   └── weatherController.js ← OpenWeatherMap + agri-advisory engine
│   │
│   ├── models\
│   │   └── User.js ← Mongoose schema: farmer user profile
│   │
│   └── middleware\
│       └── auth.js ← JWT authentication middleware
│
└── client\ ← React 19 + Vite Frontend
    ├── index.html ← SPA root HTML shell
    ├── vite.config.js ← Vite bundler + API proxy to :5000
    ├── tailwind.config.js ← Custom colour themes
    ├── postcss.config.js
    ├── package.json ← Frontend dependencies
    │
    ├── public\
    │   ├── favicon.svg
    │   ├── realistic_smart_farm.jpg ← Hero banner (day mode)
    │   └── smart_farm_thermal.jpg ← Hero banner (thermal mode)
    │
    └── src\
        ├── main.jsx ← React DOM root render
        ├── App.jsx ← Root component: state, routing, TTS
        ├── App.css ← Global CSS overrides
        ├── index.css ← Tailwind base + custom keyframes
        │
        ├── locales\
        │   └── translations.js ← i18n dictionary: en / hi / ta / te
        │
        ├── services\
        │   └── api.js ← HTTP service layer → /api/*
        │
        ├── assets\
        │   └── hero.png
        │
        └── components\
            ├── Navbar.jsx ← Language switcher, theme toggle, auth
            ├── AuthModal.jsx ← Login / Register modal
            ├── FarmerDashboard.jsx ← IoT live feed + morning briefing
            ├── FarmVisualizer3D.jsx ← Holographic 3D farm map
            ├── AiChatbot.jsx ← Voice + text Gemini AI chatbot
            ├── CropRecommendation.jsx ← NPK crop advisor form
            ├── WeatherAdvisory.jsx ← Weather + 7-day agri forecast
            ├── LeafDiseaseDetection.jsx← Leaf image disease analyser
            ├── SmartIrrigation.jsx ← Drip irrigation planner
            ├── FertilizerCalculator.jsx← NPK fertilizer dose calculator
            ├── MarketPriceProfit.jsx ← Mandi prices + profit predictor
            └── GovtSchemes.jsx ← Govt scheme browser
```

6. Core Modules in Detail

6.1 Authentication System

The authentication system is built using two standard Node.js libraries — bcryptjs for secure password hashing and jsonwebtoken for stateless session management. When a farmer registers, the plain-text password is hashed using bcrypt with a salt factor of 10 before being stored in the MongoDB Atlas users collection. No plain-text passwords are ever stored in the database under any circumstance.

When the farmer logs in, bcrypt compares the submitted password against the stored hash. On successful verification, the server generates a JWT signed with a secret key and valid for 24 hours. The React client stores this token in application state and includes it in the Authorization header of subsequent requests to protected routes. The `authenticateToken` middleware in `auth.js` verifies this token on every protected route call and rejects invalid or expired tokens with a 401 Unauthorized response.

6.2 Gemini AI Chatbot

The chatbot is the most sophisticated module in the platform and operates on a two-tier fallback architecture designed to ensure the chatbot remains functional even when internet connectivity is limited or the API key is not configured.

In the first tier, when the Gemini API key is present in the environment, the controller sends the farmer's query to the Google Gemini Flash model (`gemini-flash-latest`). It constructs a detailed system instruction defining the AI persona as "SmartAgri AI," a senior agronomist and precision farming assistant. The instruction covers crop health, pests, fungicides, soil nutrients, irrigation scheduling, Indian government schemes, and APMC mandi trends, and it includes a language directive instructing Gemini to respond in the farmer's chosen language with practical, bullet-pointed advice. Gemini returns a structured response, which is packaged with a category label and four localized suggested question chips before being sent to the React client.

In the second tier, if Gemini is unavailable, the controller uses a built-in knowledge base called `AGRONOMIST_INTENT_RESPONSES` . This is an array of intent objects covering five major farming topics: plant disease and fungal management, smart water and irrigation advisory, nutrient and fertilizer management, market trends and selling advice, and government agriculture schemes. Each intent

object contains a keyword array, a topic title, and a pre-authored detailed response in English and Tamil, with Telugu and Hindi support provided through greeting localization and generic structured fallbacks.

6.3 Crop Recommendation Engine

The crop recommendation engine uses a pure algorithmic weighted scoring system to match the farmer's input parameters against a database of nine major Indian crops: Rice (Paddy), Wheat, Cotton, Maize, Tomato, Sugarcane, Chickpea, Onion, and Groundnut. Each crop in the database has defined optimal ranges for seven parameters — Nitrogen (20 pts), Phosphorus (15 pts), Potassium (15 pts), Temperature (18 pts), Humidity (12 pts), soil pH (10 pts), and Soil Type preference bonus (10 pts). For each parameter, if the farmer's value falls within the crop's optimal range, the full points are awarded. If it falls outside, partial points are awarded proportional to proximity to the boundary. The final confidence score is clamped between 45 and 99 percent. Crops are ranked by descending confidence score and the top crop is returned as the primary recommendation, with the next two as alternatives.

6.4 IoT Soil Sensor Telemetry

The IoT module simulates a real-time wireless soil sensor node identified as "AGRI-NODE-01-NASHIK." The virtual sensor node maintains a persistent in-memory state object containing soil moisture percentage, soil temperature in Celsius, ambient temperature, ambient humidity, nitrogen, phosphorus, and potassium levels in kg per hectare, soil pH, electrical conductivity in dS/m, pump status, and auto-mode flag.

Every time the React dashboard calls the telemetry endpoint (on a polling interval), the controller adds a small random jitter value to the soil moisture to simulate natural field fluctuations. If the pump is active, moisture increases at 0.8% per poll. If the pump is off, it decreases slowly at 0.1% per poll. The auto-mode logic activates the pump when moisture falls below 25% and deactivates it when it rises above 55%, exactly mimicking a real-world drip irrigation controller. The farmer can manually override the pump through the dashboard toggle button.

6.5 Weather Advisory Engine

The weather controller integrates with the OpenWeatherMap API to fetch live current weather and five-day forecast data for any Indian city. It calls two OpenWeatherMap endpoints — one for current conditions and one for the extended forecast. The controller extracts temperature, feels-like temperature, humidity, wind speed, cloud cover, atmospheric pressure, and general weather

condition, then maps these values into a seven-day agriculture-specific forecast array. Each day entry includes maximum and minimum temperatures, rainfall probability, overall condition, and two boolean flags: `suitableForSpraying` (true when rain chance is below 35%) and `suitableForHarvesting` (true when rain chance is below 20%).

The controller also generates three smart advisory messages based on current conditions — an evaluation of the ideal spraying window, a medium-urgency fungal blight risk warning when high humidity is forecasted, and an irrigation scheduling advisory before predicted rainfall. When the API key is not configured, the controller falls back to a regional agri-meteorology model with hardcoded seasonal baseline values.

6.6 3D Farm Visualizer

The 3D Farm Visualizer is built entirely with CSS 3D perspective transforms and Framer Motion animations, without any dependency on WebGL or Three.js. This architectural choice ensures the visualizer works smoothly even on low-end Android smartphones that do not have GPU acceleration for WebGL. The visualizer renders a top-down perspective view of a smart farm with four interactive holographic hotspot nodes: the IoT Sensor Station, the AI Drone surveillance zone, the Micro-Drip irrigation system area, and Sector A crop field. Each hotspot pulses with a neon glow animation and can be clicked to reveal a detailed HUD panel with live telemetry data. The farmer can toggle between two camera modes — a Daylight Drone View showing the realistic farm layout, and a Thermal NDVI Heatmap view showing a colour-coded vegetation health overlay. All labels and HUD content are fully localized in all four languages.

6.7 Multilingual i18n System

The internationalization system is built as a central translation dictionary in `translations.js`. This file exports a single JavaScript object where the top-level keys are language codes: `en`, `hi`, `ta`, and `te`. Each language object contains key-value pairs for every piece of UI text — navigation labels, tab names, feature card titles and descriptions, IoT metric labels, chatbot placeholder text, footer credits, and status messages. The App component holds the active language in its React state and passes it as a prop called `language` to every child component.

For voice capabilities, the platform uses the browser's native Web Speech API. Text-to-speech is handled by `SpeechSynthesisUtterance`, where the `lang` property is set to the matching IETF locale tag: `en-IN`, `hi-IN`, `ta-IN`, or `te-IN`. Voice input for the chatbot is handled by `webkitSpeechRecognition`, also configured with the matching locale so the browser correctly transcribes speech in native script.

7. Technology Stack

Backend Technologies

Node.js (v24.14.0) is the JavaScript runtime environment that powers the entire backend. It uses the ES Module system (`type: "module"`) allowing the use of modern `import/export` syntax throughout the server codebase. **Express.js (v4.21.2)** is used as the REST API framework, providing routing, middleware handling, JSON body parsing, and request/response lifecycle management. **Mongoose (v8.9.5)** serves as the Object Document Mapper for MongoDB, providing schema validation, query building, and connection management. **MongoDB Atlas** is the cloud-hosted database service where the `smartagri` database and the `users` collection reside.

bcryptjs (v2.4.3) handles all password hashing operations using the bcrypt algorithm with a configurable salt factor, ensuring passwords are securely stored. **jsonwebtoken (v9.0.2)** issues and verifies JWT tokens for stateless authentication sessions, eliminating the need for server-side session storage. **Multer (v1.4.5)** handles multipart file uploads, used specifically for the leaf disease detection feature where farmers upload crop images. **dotenv (v16.4.7)** loads environment variables from the `.env` file into `process.env` at server startup, keeping all secrets out of the source code.

Frontend Technologies

React 19 (v19.2.8) is the latest major version of the React framework, used for building the entire single-page application UI with component-based architecture. **Vite (v8.3.0)** serves as both the development server and the production bundler, compiling the entire React application into a single optimized 362 kilobyte JavaScript bundle and a 52 kilobyte CSS bundle, achieving this in just 3 seconds. **Tailwind CSS (v3.4.19)** provides utility-first CSS classes that define the two complete visual themes — Light Nature green and Cyber Neon Dark.

Framer Motion (v13.5.0) powers all animations — card entrance transitions, hotspot pulsing effects, modal fade-ins, and the 3D perspective animations of the farm visualizer. **Lucide React (v1.49.0)** provides a consistent, lightweight SVG icon library used across all components. **canvas-confetti (v1.9.4)** creates celebration particle effects on successful crop recommendation results. The **Web Speech API** is a browser-native API requiring no additional dependency — it handles both voice input (speech-to-text) and voice output (text-to-speech) in all four Indian language locales.

8. Database Schema

The MongoDB `smartagri` database contains one collection called `users`. Each document in this collection represents one registered farmer on the platform. The schema is defined and enforced by Mongoose in the `User.js` model file.

```
// MongoDB Document Structure — users collection
{
  name:               String,    // Required. Farmer's full name (trimmed)
  email:              String,    // Required. Unique. Stored lowercase (login key)
  password:           String,    // Required. bcrypt hashed — never plain text
  phone:              String,    // Optional. Contact number

  location: {
    state:            String,    // Default: "Maharashtra"
    district:         String,    // Default: "Nashik"
    village:          String     // Default: "Panchavati"
  },

  farmDetails: {
    sizeInAcres:      Number,    // Default: 4.5
    soilType:         String,    // Default: "Black Soil"
    primaryCrop:      String,    // Default: "Tomato"
    irrigationType:   String     // Default: "Drip Irrigation"
  },

  preferredLanguage: String,    // Enum: ['en', 'hi', 'mr', 'ta', 'te'] — Default: 'en'
  createdAt:         Date       // Default: Date.now (auto-set at document creation)
}
```

Auto-Seeding: On the first successful connection to MongoDB Atlas, the server automatically seeds one default farmer profile — "Ramesh Patel" at farmer@smartagri.com, password "farmer123" (bcrypt hashed), a 4.5-acre tomato farm in Nashik, Maharashtra with preferred language set to Tamil. This ensures the Atlas collections panel always shows populated data for evaluation and demonstration.

9. Multilingual Support

The SmartAgri platform provides complete, feature-identical support for four Indian languages. Every layer of the application — from the MongoDB schema to the Express controllers, the React components, and the browser's speech engine — is configured to work seamlessly in all four languages.

English (en / en-IN): The default language. All UI labels, chatbot system prompts, agronomic knowledge base responses, voice TTS readout, STT voice input, morning audio briefing, 3D holographic hotspot labels, and suggested chatbot question chips are available in English. The morning briefing begins with "Good morning, Farmer!" and the chatbot greeting is "Hello Farmer Brother!"

Hindi (hi / hi-IN): All UI text is translated into Hindi. The Gemini AI chatbot is instructed to respond in simple, helpful Hindi for an Indian farmer. The TTS and STT use the `hi-IN` locale code. The fallback chatbot greeting is "नमस्ते किसान भाई! 🙏" and suggested questions cover topics like tomato blight, drip irrigation timing, wheat DAP requirements, and PM-KUSUM subsidy application.

Tamil (ta / ta-IN): Complete Tamil localization across all 12 React components. The Gemini AI chatbot responds in clear, practical Tamil. The TTS and STT use `ta-IN`. The agronomic knowledge base provides full pre-authored Tamil responses for all five intent categories (disease, irrigation, fertilizer, market, schemes). The morning briefing begins with "வணக்கம் உழவரே!" and the chatbot greeting is "வணக்கம் உழவரே! 🙏". Four Tamil suggested questions cover tomato blight control, drip irrigation timing, paddy DAP dosage, and PM-KUSUM solar pump subsidy.

Telugu (te / te-IN): Complete Telugu localization implemented across all layers. The Navbar shows "తెలుగు (Telugu)" and the 3D brand badge updates to "3D తెలుగు". The Gemini AI chatbot is instructed to respond in clear, practical Telugu. The TTS and STT use `te-IN` for native script transcription. The morning audio briefing begins "నమస్కారం రైతు సోదరా!". All four 3D holographic hotspot labels, the two camera view toggle buttons, and all IoT panel labels are localized in Telugu script. The chatbot greeting is "నమస్కారం రైతు సోదరా! 🙏" and four Telugu suggested questions cover tomato leaf spot disease, drip irrigation timing, rice crop DAP and Urea requirements, and PM-KUSUM solar pump subsidy application.

10. End-to-End Application Workflow

The complete end-to-end user journey through the SmartAgri platform works as follows. A farmer opens the application in any modern web browser and is greeted by the SmartAgri landing page styled in the active theme. The farmer first selects their preferred language from the dropdown in the navigation bar, and the entire UI immediately re-renders in the chosen language without a page reload.

The farmer then clicks the login button and either registers a new account by entering their name, email, password, and farm details, or logs in with existing credentials. Upon successful login, the JWT token is issued and the main Farmer Dashboard is displayed. The dashboard shows a live IoT sensor panel with real-time readings for soil moisture, soil temperature, NPK levels, pH, and electrical conductivity, all updating on a polling interval. The morning audio briefing button plays a spoken agricultural summary in the farmer's language using the browser's text-to-speech engine.

From the dashboard, the farmer can navigate to any of the eight feature modules using the tab-based navigation. In the **AI Chatbot** tab, the farmer can type or speak a farming question. The query is sent to the backend, which calls Gemini AI and returns a structured multilingual response in under two seconds. The chatbot displays four suggested follow-up questions that the farmer can click to ask next, all in the farmer's native language.

In the **Crop Recommendation** tab, the farmer fills in their current soil test values — nitrogen, phosphorus, potassium, and pH — along with ambient temperature, humidity, expected rainfall, and soil type. On clicking the recommend button, the weighted scoring algorithm runs on the server and returns the best-matching crop with a confidence percentage, along with the next two alternatives. Each result includes growth duration, expected yield, water requirement, and expert cultivation tips.

In the **Weather Advisory** tab, the farmer enters their nearest city name. The system fetches live weather data and displays current conditions alongside a seven-day forecast. Each forecast day shows the maximum and minimum temperature, overall weather condition, rainfall probability, and whether that day is suitable for spraying or harvesting. Three agriculture-specific advisory cards provide actionable guidance about the spraying window, fungal disease alerts, and irrigation scheduling.

In the **Leaf Disease Detection** tab, the farmer can drag and drop or browse to upload a photo of a diseased leaf. The image is sent to the backend disease controller, which analyses it and returns a

disease name, confidence level, severity assessment, and a step-by-step treatment plan with both organic and chemical remedies.

In the **Smart Irrigation** tab, the farmer enters their crop type, field area, current soil moisture, and local weather conditions. The irrigation planner calculates the optimal drip schedule — how many irrigation cycles per day, the duration of each cycle, and the total water volume required — to maintain field capacity without waterlogging or root hypoxia.

In the **Fertilizer Calculator** tab, the farmer selects their crop, growth stage, and known soil nutrient deficiencies. The calculator returns a complete NPK dose plan with recommended quantities of Urea, DAP, and MOP per acre, split into basal and top-dress doses with the application timing for each stage.

In the **Market Prices** tab, the farmer sees current APMC mandi wholesale prices for major Indian crops. They can also enter crop, acreage, and expected yield to get a profit prediction for the upcoming harvest season based on projected market price trends.

In the **Government Schemes** tab, the farmer browses a curated list of active central and state government agriculture schemes. Each scheme card shows the scheme name, benefit amount or subsidy percentage, eligibility criteria, application method, and a direct link to the official government portal for PM-KISAN, PM-KUSUM, PM Fasal Bima Yojana, and the Soil Health Card scheme.

In the **3D Farm Visualizer**, the farmer explores an interactive holographic map of their smart farm. Four hotspot nodes — IoT Station, AI Drone, Micro-Drip System, and Sector A Crops — can be clicked to reveal live telemetry HUD panels. The farmer can switch between Daylight Drone View and Thermal NDVI Heatmap View using the toggle buttons, all labelled in the active language.

11. Environment Configuration

All sensitive configuration values are stored in a `.env` file located inside the `server` directory. This file is excluded from version control via `.gitignore` and is never committed to the repository. The server reads all values at startup using the dotenv library and accesses them throughout the application via `process.env.VARIABLE_NAME`. No credentials are hardcoded anywhere in the source code.

```
# d:\Smartagri\server\.env

# MongoDB Atlas Connection String
MONGODB_URI=mongodb+srv://Smartagri:<password>@smartagri.6b3xbng.mongodb.net/?
appName=Smartagri

# Google Gemini AI API Key (from Google AI Studio)
GEMINI_API_KEY=<your_google_gemini_api_key>

# OpenWeatherMap API Key (from openweathermap.org)
WEATHER_API_KEY=<your_openweathermap_api_key>

# Express Server Port (default: 5000)
PORT=5000
```

Security Note: The MONGODB_URI contains the Atlas database username and password. The GEMINI_API_KEY grants access to Google AI billing. The WEATHER_API_KEY is rate-limited per plan. All three must remain exclusively in the .env file and must never be exposed in client-side code or committed to any public repository.

12. Key Design Decisions

100% Pure MERN Stack: The decision to use a single unified MERN stack was made to ensure the entire application runs on one consistent language — JavaScript — across all four layers of the system. This eliminates context-switching overhead between languages, reduces onboarding time for new developers, and keeps the deployment model simple: two processes (Node.js backend and Vite/Node.js frontend) running on one machine or server.

Two-Tier AI Fallback Architecture: The chatbot's two-tier design — Gemini AI as the primary engine and the built-in knowledge base as the fallback — is a deliberate resilience engineering choice. By maintaining a self-contained agronomic knowledge base, the chatbot remains fully functional even in rural areas with intermittent internet connectivity or when the Gemini API is temporarily unavailable. This is critical for the target users — farmers in semi-urban and rural India who depend on the platform for time-sensitive farming decisions.

In-Memory IoT Simulation: The decision to use a pure Node.js in-memory IoT sensor simulation rather than real hardware was made to allow the platform to be fully demonstrable in any environment without requiring physical sensor equipment. The simulation is realistic enough — with natural jitter, auto pump logic based on moisture thresholds, and realistic sensor value ranges — to serve as a meaningful proof-of-concept for a real deployment that would replace the simulated values with actual sensor readings from field-deployed nodes communicating over MQTT or 4G LTE-M.

Weighted Scoring for Crop Recommendation: The transparent, algorithmic weighted scoring system was chosen over a black-box machine learning model because it is fully explainable and auditable. A farmer, an agriculture officer, or a report evaluator can clearly understand exactly why a particular crop was recommended — which parameters matched, which did not, and what the confidence score means. This aligns with responsible AI principles in agriculture, where decisions directly affect a farmer's livelihood.

CSS 3D Transforms Instead of WebGL: The decision to use CSS 3D transforms and Framer Motion for the farm visualizer rather than WebGL or Three.js was driven by device compatibility. Many farmers in rural India access the internet on mid-range Android smartphones that do not support WebGL reliably. CSS 3D transforms are supported by virtually every modern browser regardless of GPU availability, making the interactive 3D experience universally accessible without performance degradation.

Browser-Native Web Speech API: Using the browser's built-in Web Speech API for both voice input and text-to-speech with no additional SDK or paid service keeps the frontend bundle size small, eliminates any proprietary voice API costs, and avoids adding network round-trips for voice processing. The native browser speech engine supports all four Indian language locales (en-IN, hi-IN, ta-IN, te-IN) natively on Android and desktop Chrome, which is the dominant browser among the target demographic.

MongoDB Atlas with Graceful In-Memory Fallback: Configuring the MongoDB connection with a 5-second timeout and an automatic fallback to in-memory persistence ensures the application never crashes due to a database connectivity issue — a common scenario when IP whitelisting is misconfigured on the Atlas cluster. The platform degrades gracefully, running all features except persistent user data storage, which allows continuous development and demonstration without requiring an active Atlas connection at all times.

13. Conclusion

SmartAgri is a comprehensive, production-ready AI advisory platform that demonstrates the full capability of the MERN stack applied to the domain of precision agriculture. By combining Google Gemini AI for multilingual natural language advisory, OpenWeatherMap for live agri-meteorology, a transparent algorithmic crop recommendation engine, a realistic IoT sensor simulation, and a visually engaging 3D farm visualizer — all within a single unified web application — the project delivers a complete digital farming assistant that serves farmers across four major Indian languages.

The platform is architecturally sound, with clear separation between the client, server, and database layers. It is secure, using bcrypt password hashing and JWT-based authentication. It is resilient, with two-tier AI fallback and database connection fallback ensuring uninterrupted operation in low-connectivity environments. It is inclusive, with full feature parity across English, Hindi, Tamil, and Telugu, including native script voice input and output.

The codebase is organized in a clean, maintainable structure with each feature module isolated in its own controller, component, and route definition. The environment configuration follows security best practices with all secrets stored exclusively in environment variables. The production build compiles to a compact 362 KB JavaScript bundle, making the application lightweight enough to load quickly even on slow mobile internet connections.

SmartAgri represents a practical, scalable blueprint for how AI, cloud databases, and modern web technologies can be combined to create meaningful, accessible tools for the agricultural sector — directly empowering the farmers who feed the nation.
